

Questão 01 – Busca em amplitude/largura, Busca em profundidade e Busca iterativa em profundidade

Um agente deve se deslocar entre salas de um prédio. Cada sala é representada por uma letra. As conexões representam movimentos permitidos entre salas. Todas as ações possuem custo unitário. A tabela abaixo apresenta o grafo do problema.

Sala	Salas vizinhas (ordem obrigatória)
A	B, C, D
B	E, F
C	G, H
D	I
E	J
F	K, L
G	M
H	N, O
I	P
J	-
K	Q
L	-
M	R
N	-
O	S
P	-
Q	-
R	-
S	-

Estado inicial: A

Estado objetivo: S

Formular o problema acima como um problema de busca em espaços de estados. A resposta deve conter: a) definição formal do estado; b) estado inicial; c) teste de objetivo; d) função sucessora; e) função de custo; f) representação escolhida em Python. Em seguida, resolva o problema utilizando os seguintes algoritmos:

1. Busca em amplitude/largura (Breadth-First Search);
2. Busca em profundidade (Depth-First Search);
3. Busca iterativa em profundidade (Iterative Deepening Search).

Para cada algoritmo, apresentar:

- a) ordem completa de expansão dos nós;
- b) conteúdo da fronteira após cada expansão;
- c) árvore parcial de busca;
- d) caminho solução encontrado;
- e) profundidade da solução;
- f) custo da solução;
- g) quantidade de nós gerados;
- h) quantidade de nós expandidos.

Preencher uma tabela semelhante ao modelo abaixo.

Passo	Nó expandido	Conteúdo da fronteira
1	A	[B,C,D]
2	B	[...]
3	...	...

Implementar os três algoritmos em Python respeitando os seguintes critérios: NÃO utilizar bibliotecas externas, bibliotecas prontas de grafos e implementações encontradas na internet. Todo o código deve ser autoral.

Experimento adicional: modificar a ordem dos sucessores de exatamente dois nós do grafo. Exemplo:

• alterar:

$C : G, H$

para:

$C : H, G$

Após a modificação, 1) executar novamente os três algoritmos; 2) comparar os resultados obtidos; e 3) analisar o impacto da ordem dos sucessores.

Responder as seguintes questões:

- a) O caminho solução mudou?
- b) A quantidade de nós expandidos mudou?
- c) Algum algoritmo foi mais sensível à ordem dos sucessores?
- d) Qual algoritmo apresentou maior consumo de memória?
- e) Qual algoritmo encontrou a solução mais rapidamente?

Por fim, comparar os algoritmos considerando: a) completude; b) otimalidade; c) complexidade de tempo; d) complexidade de espaço; e) dependência da ordem dos sucessores; f) comportamento em árvores profundas; g) adequação para problemas grandes.

Questão 02 – Busca Gulosa pela Melhor Escolha e A*

Considerar o seguinte problema de navegação em um ambiente parcialmente bloqueado: um agente deve sair da posição A e alcançar a posição T. Cada aresta possui um custo associado. Além disso, cada estado possui um valor heurístico $h(n)$, que representa uma estimativa da distância restante até o objetivo. O grafo do problema é apresentado abaixo.

Nó	Sucessores (custo)	$h(n)$
A	B(2), C(4), D(3)	10
B	E(3), F(5)	8
C	G(4), H(6)	7
D	I(2)	9
E	J(4)	6
F	K(3), L(5)	5
G	M(6)	6
H	N(3), O(4)	4
I	P(5)	7
J	Q(4)	5
K	R(3)	3
L	-	6
M	S(2)	3
N	-	4
O	T(5)	1
P	-	8
Q	-	4
R	T(4)	2
S	T(3)	1
T	-	0

Estado inicial: A

Estado objetivo: T

Modelar o problema como um problema de busca em espaço de estados. Apresentar: a) representação dos estados; b) estado inicial; c) teste de objetivo; d) função sucessora; e) função de custo; f) interpretação da heurística $h(n)$. Em seguida, resolver o problema manualmente usando o algoritmo Greedy Best-First Search utilizando o valor heurístico:

$$f(n) = h(n)$$

Apresentar:

- a) ordem de expansão dos nós;
- b) conteúdo da fronteira após cada expansão;
- c) valores heurísticos utilizados;
- d) caminho solução encontrado;
- e) custo final da solução;

- f) quantidade de nós gerados;
- g) quantidade de nós expandidos.

Resolver o mesmo problema utilizando o algoritmo A* com:

$$f(n) = g(n) + h(n)$$

onde:

- $g(n)$: custo acumulado do caminho;
- $h(n)$: heurística fornecida;
- $f(n)$: custo estimado total.

Para cada expansão, apresentar:

- nó expandido;
- valor de $g(n)$;
- valor de $h(n)$;
- valor de $f(n)$;
- conteúdo da fronteira ordenada;
- caminho parcial até o nó.

Preencher tabelas no seguinte formato:

Passo	Nó	$g(n)$	$h(n)$	$f(n)$
1	A	0	10	10
2	B	2	8	10
3	...	...	...	...

Implementar os algoritmos *Busca Gulosa pela Melhor Escolha* e *Busca A** em Python respeitando os seguintes critérios: NÃO utilizar bibliotecas externas, bibliotecas prontas de grafos e implementações encontradas na internet. Todo o código deve ser autoral.

Modificar os valores heurísticos de exatamente três nós do grafo. Após isso, 1) execute novamente os algoritmos, 2) compare os resultados; e 3) analise o impacto da heurística.

Responder as seguintes questões:

- A solução encontrada mudou?
- O custo da solução mudou?
- A busca gulosa encontrou solução ótima?
- O A* encontrou solução ótima?
- Qual algoritmo expandiu menos nós?
- Qual algoritmo foi mais sensível à heurística?

Por fim, comparar **Greedy Best-First Search** e **A*** considerando: a) completude; b) otimalidade; c) consumo de memória; d) dependência da heurística; e) custo computacional; f) qualidade das soluções; g) comportamento em grafos grandes.

Questão 03 – Busca Local em Problema das N-Rainhas

Considerar o problema das 8-Rainhas. O objetivo é posicionar 8 rainhas em um tabuleiro 8×8 de forma que nenhuma rainha ataque outra. Duas rainhas entram em conflito quando estão i) na mesma linha, ii) na mesma coluna e iii) na mesma diagonal. Nesta atividade, cada estado será representado por um vetor:

$$[s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8]$$

onde:

$$s_i = \text{linha da rainha na coluna } i$$

Exemplo:

$$[4, 2, 7, 3, 6, 8, 5, 1]$$

significa:

- coluna 1 $\rightarrow$ linha 4;
- coluna 2 $\rightarrow$ linha 2;

- ...
- coluna 8 $\rightarrow$ linha 1.

Iniciar a busca a partir do estado:

$$[1, 1, 1, 1, 1, 1, 1, 1]$$

Considerar a seguinte função heurística:

$$h(s) = \text{número total de pares de rainhas em conflito}$$

Objetivo:

$$\min h(s)$$

Um estado solução ocorre quando:

$$h(s) = 0$$

Modelar o problema como um **problema de busca local**. Apresentar: a) representação do estado; b) definição de vizinho; c) função de avaliação; d) critério de parada; e) interpretação da superfície de busca. Em seguida, resolver o problema manualmente utilizando o **algoritmo Hill-Climbing**, onde um vizinho é obtido movendo exatamente UMA rainha para outra linha da MESMA coluna. A solução deve apresentar:

- estado atual;
- todos os vizinhos avaliados em cada iteração;
- valor de $h(s)$ para cada vizinho;
- estado escolhido;
- explicação da escolha realizada;
- número total de iterações;
- estado final encontrado.

A resposta deve conter tabelas semelhantes ao modelo:

Iteração	Estado	$h(s)$
0	$[1, 1, 1, 1, 1, 1, 1, 1]$	28
1	...	...
2	...	...

Além disso, para cada iteração, mostrar pelo menos os 5 melhores vizinhos gerados.

Durante a execução do Hill-Climbing, identificar se ocorreu máximo local, platô, pico estreito ou solução global. Caso algum desses itens ocorra, explicar:

- por que ocorreu;
- como isso afetou a busca;
- como poderia ser evitado.

Implementar uma versão com **Random Restart Hill-Climbing**:

- Executar o algoritmo 20 vezes;
- Cada execução deve iniciar em um estado aleatório;
- Registrar:
 - número de passos;
 - valor final de $h(s)$;
 - se encontrou solução ou não.

Apresentar a seguinte tabela:

Execução	Estado inicial	Passos	$h(s)$ final
1	...	...	...
2	...	...	...

Implementar o algoritmo **Simulated Annealing** usando a seguinte função de aceitação:

$$P = e^{-\Delta E/T}$$

onde:

- ΔE : aumento do número de conflitos;
- T : temperatura atual.

Apresentar:

- valor inicial da temperatura;
- política de resfriamento utilizada;
- exemplos de movimentos piores aceitos;
- comparação com Hill-Climbing simples;
- quantidade de soluções válidas encontradas.

Implementar os algoritmos *Hill-Climbing*, *Random Restart Hill-Climbing* e *Simulated Annealing* em Python respeitando os seguintes critérios: NÃO utilizar bibliotecas externas, bibliotecas prontas de otimização e implementações encontradas na internet. Todo o código deve ser autoral.

Por fim, comparar os algoritmos considerando: a) qualidade das soluções; b) velocidade de convergência; c) sensibilidade ao estado inicial; d) capacidade de escapar de máximos locais; e) custo computacional; f) estabilidade dos resultados.

Questão 04 – CSP e Otimizações de Busca

Um hospital precisa montar automaticamente a escala de plantões de um pequeno grupo de médicos. Os turnos são:

$$T_1, T_2, T_3, T_4, T_5, T_6$$

Os médicos disponíveis são:

$$\{A, B, C, D\}$$

Cada turno deve possuir exatamente um médico.

Restrições que devem ser consideradas:

- O mesmo médico NÃO pode trabalhar em turnos consecutivos.
- O médico *A* não pode trabalhar em:

$$T_3$$

- O médico *B* deve trabalhar em pelo menos um turno entre:

$$T_1, T_2$$

- O médico *C* não pode trabalhar simultaneamente em:

$$T_2 \text{ e } T_5$$

- O médico *D* pode trabalhar no máximo dois turnos.

Modelar o problema como um **Constraint Satisfaction Problem (CSP)**. Apresentar: a) conjunto de variáveis; b) domínio de cada variável; c) restrições unárias; d) restrições binárias; e) restrições globais; f) representação do grafo de restrições. Em seguida, resolver o problema manualmente utilizando **Backtracking** e apresentar:

- árvore parcial da busca;
- ordem de atribuição das variáveis;
- conflitos encontrados;
- estados descartados;
- retrocessos realizados;
- solução final encontrada.

Apresentar a tabela:

Passo	Variável atribuída	Estado parcial
1	$T_1 = A$	$\{T_1 = A\}$
2	$T_2 = A$	conflito
3	$T_2 = B$	$\{T_1 = A, T_2 = B\}$

Resolver o problema utilizando **MRV (Minimum Remaining Values)** e **Degree Heuristic** e explicar:

- qual variável foi escolhida em cada passo;
- por que ela foi escolhida;
- como MRV reduziu o espaço de busca;
- como Degree influenciou a busca.

Implementar **Forward Checking** e apresentar:

- redução dos domínios após cada atribuição;
- domínios eliminados;
- momento em que inconsistências foram detectadas;
- comparação com backtracking simples.

Implementar **Backjumping** e apresentar:

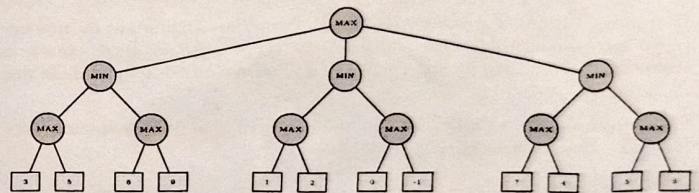
- variáveis responsáveis pelos conflitos;
- saltos realizados;
- diferenças para backtracking tradicional;
- redução observada na árvore de busca.

Implementar os algoritmos *Backtracking*, *Backtracking + MRV*, *Backtracking + MRV + Degree*, *Forward Checking* e *Backjumping* em Python respeitando os seguintes critérios: NÃO utilizar bibliotecas externas, bibliotecas prontas de CSP, OR-Tools e implementações encontradas na internet. Todo o código deve ser autoral.

Por fim, comparar os algoritmos considerando: a) número de estados explorados; b) número de retrocessos; c) velocidade de execução; d) consumo de memória; e) facilidade de implementação; f) eficiência prática.

Questão 05 – Minimax e Poda Alpha-Beta

Considerar a seguinte árvore de jogo para um jogo determinístico de dois jogadores. O jogador MAX realiza a primeira jogada na raiz da árvore. Os valores nas folhas representam a utilidade final do jogo.



Os valores terminais são apresentados abaixo, da esquerda para a direita:

$$[3, 5, 6, 9, 1, 2, 0, -1, 7, 4, 5, 6]$$

A ordem de expansão dos filhos deve seguir obrigatoriamente da esquerda para a direita. Explicar:

- objetivo do algoritmo Minimax;
- diferença entre nós MAX e MIN;
- conceito de utilidade;
- propagação de valores na árvore;
- hipótese de adversário perfeito;
- objetivo da poda Alpha-Beta.

Executar passo a passo o algoritmo Minimax na árvore apresentada e apresentar: a) valores calculados em cada nó; b) ordem de expansão; c) propagação dos valores; d) decisão tomada por MAX; e) caminho escolhido pelo algoritmo; f) árvore parcialmente preenchida.

Apresentar a tabela:

Nó	Tipo	Valor Minimax
A	MAX	...
B	MIN	...
C	MAX	...

Executar a **Poda Alpha-Beta** utilizando a MESMA ordem de expansão. Apresentar:

- valores de α e β em cada passo;
- momento em que ocorreram podas;
- quais ramos foram podados;
- motivo matemático da poda;
- quantidade de nós NÃO explorados.

Apresentar a tabela:

Passo	Nó	α	β	Poda?
1	A	...	...	não
2	B	...	...	sim

Alterar a ordem de expansão dos filhos de forma a maximizar o número de podas. Em seguida, discutir:

- Quantas podas ocorreram antes?
- Quantas podas ocorreram depois?
- Por que a nova ordem melhora a eficiência?
- Qual a relação entre ordenação e desempenho?

Considerar agora que a árvore é muito grande e que a busca deve parar na profundidade 2. Os seguintes valores heurísticos devem ser usados nos nós não-terminais da profundidade limite:

[4, 7, 2, 5, 6, 1]

Responder:

- executar Minimax limitado;
- utilizar os valores heurísticos;
- comparar a decisão com Minimax completo;
- discutir possíveis erros causados pela heurística.

Implementar os algoritmos *Minimax*, *Alpha-Beta* e *Minimax com profundidade limitada*, em Python respeitando os seguintes critérios: NÃO utilizar bibliotecas externas, bibliotecas prontas de jogos, e implementações encontradas na internet. Todo o código deve ser autoral.

Comparar Minimax e Alpha-Beta considerando: a) número de nós explorados; b) quantidade de podas; c) custo computacional; d) consumo de memória; e) impacto da ordenação dos movimentos; f) qualidade das decisões.

Em um experimento adicional, modificar os valores de exatamente 3 folhas da árvore. Após isso:

- executar novamente Minimax;
- executar novamente Alpha-Beta;
- verificar se a decisão final mudou;
- analisar a sensibilidade do algoritmo às utilidades.

Questão 06 – Monte Carlo Tree Search (MCTS)

Considerar o jogo Connect-4 simplificado abaixo. O jogador vermelho deve decidir qual coluna jogar. O estado atual é:

A	V		
V	A		

Legenda: V: vermelho e A: amarelo. As ações possíveis são:

c_1, c_2, c_3, c_4

Para MCST, explicar detalhadamente:

- seleção;
- expansão;
- simulação (rollout);
- retropropagação.

Explicar também:

- papel do número de visitas;

- papel do número de vitórias;
- diferença entre exploração e explotação.

Executar 10 iterações do algoritmo MCTS. Em cada iteração apresentar:

- caminho selecionado;
- nó expandido;
- resultado do rollout;
- atualização de:

$$N(s), W(s)$$

- árvore parcial construída.

Utilizar:

$$UCT(j) = \frac{w_j}{n_j} + C \sqrt{\frac{\ln N}{n_j}}$$

com:

$$C = 1.4$$

e calcular:

- valor de UCT para cada filho;
- nó selecionado;
- influência do termo de exploração;
- influência do termo de explotação.

Apresentar a seguinte tabela:

Jogada	Visitas	Vitórias	UCT
c1	3	2	...
c2	5	4	...
c3	1	1	...

Executar novamente o algoritmo utilizando $C = 0.1$ e depois $C = 3.0$ e comparar:

- quantidade de exploração;
- diversidade de jogadas;
- estabilidade das decisões;
- qualidade das jogadas encontradas.

Implementar dois tipos de rollout:

- rollout totalmente aleatório;
- rollout semi-guloso:
 - priorizar jogadas centrais;
 - bloquear vitória imediata do adversário.

e comparar:

- qualidade das decisões;
- velocidade de convergência;
- número de simulações necessárias.

Implementar *seleção*, *expansão*, *rollout*, *retropropagação* e *cálculo de UCT* em Python respeitando os seguintes critérios: NÃO utilizar bibliotecas externas, bibliotecas prontas de jogos e implementações encontradas na internet. Todo o código deve ser autoral.

Por fim, comparar: a) rollout aleatório vs semi-guloso; b) diferentes valores de C ; c) número de iterações; d) estabilidade das decisões.